讲座笔记

常驻智能体是怎样工作的

How always-on agents work ・Lee Robinson 斯坦福 CS146S 讲座

整理：EstreWolf（@EstreWolf）

2026 年 10 月 10 日



原视频作者	Lee Robinson（X：@leerob），SpaceXAI
时间	讲座 2026-10-08（Stanford CS146S）；X 发布 2026-10-10 04:06（UTC+8）
视频时长	45:17（英文；文字依据为 X 平台英文字幕）
原帖链接	https://x.com/leerob/status/2108650243365736855
版权说明	讲座内容与幻灯片版权归原作者所有；本笔记为学习整理，截图均标注视频时间

EstreWolf

目录

1 导读 1

1.1 这是一段什么视频 1
1.2 怎样读这份笔记 1
1.3 本章小结 1

2 从终端里的循环到常驻智能体 2

2.1 起点：第一周写过的循环 2
2.2 常驻智能体多了什么 2
2.3 开发者与 AI 协作的四个阶段 3
2.4 从逐步批准到随时纠偏 3
2.5 模型越强，界面越简单 4
2.6 本章小结 5

3 模型发生了哪些变化 5

3.1 六项新能力 5
3.2 智能体知道的东西都放在文件里 6
3.3 本章小结 7

4 常驻智能体的内部结构 7

4.1 没事的时候什么都不运行 7
4.2 什么会唤醒它 8
4.3 智能体的电脑 8
4.4 系统的三个部分 9
4.5 为什么 agent loop 放在服务器上 10
4.6 点击“发送”之后发生了什么 11
4.7 持久化工作流与任务队列 11
4.8 新消息的处理规则 12
4.9 本章小结 12

5 Harness：工具、界面与安全 13

5.1 只有一个工具负责和你说话 13
5.2 工具配上专用界面 14
5.3 工具详情按需加载 14
5.4 先试最便宜的办法 15
5.5 使用电脑的三种方式 16
5.6 一个主智能体，多个助手 17
5.7 设计工具的几条经验 17
5.8 安全机制 18
5.9 本章小结 19

6 上下文工程 19

6.1 目标：让上下文尽量少 19
6.2 提示词开头保持不变 19
6.3 缓存未命中当作 bug 处理 20

6.4 保持主上下文精简 . . . 21
6.5 趁你离开时做压缩 . . . 22
6.6 记忆分层 . . . 22
6.7 本章小结 . . . 24

7 未来方向 . . . 24
7.1 为产品训练模型 . . . 24
7.2 产品与模型一起改进 . . . 25
7.3 删掉产品 . . . 25
7.4 6–12 个月的预期与开放问题 . . . 26
7.5 新的基础组件 . . . 27
7.6 讲者的三点建议 . . . 27
7.7 本章小结 . . . 28

8 总结与延伸 . . . 28
8.1 一张图看全部结构 . . . 28
8.2 从第一周的循环到常驻智能体：每一层解决什么问题 . . . 29
8.3 几条可迁移的原则 . . . 29
8.4 延伸思考 . . . 29

参考资料 . . . 30

1　导读

1.1　这是一段什么视频

视频是 Lee Robinson 于 2026 年 10 月 8 日在斯坦福 CS146S 课堂上的讲座录像，时长 45 分 17 秒，英文。题目是“How always-on agents work”：常驻、主动、持续运行的智能体（always-on agent）是怎样搭起来的。他在 X 上发布视频时附了六个章节时间戳 [1]，本笔记的章节与之对应。

讲座以 SpaceXAI 的 Grok Bot 为实例。Lee 开场说明，其中部分内容是 SpaceX 仍在推进的工作（00:14–00:19）。课程第一周，学生已经写过一个最简单的 harness：模型在循环里调用工具。这场讲座从那个循环出发，逐层加上服务器、云端电脑、持久化工作流、消息路由、记忆和上下文管理，说明一个能连续工作数周的智能体需要哪些部件。视频在 45:12 结束于“that's all I have today”，议程里的 Q&A 不在录像中。

资料补充｜讲者、课程与产品

- Lee Robinson 2025 年 7 月加入 Cursor，第一年做开发者体验、文档和产品营销；2026 年 6 月转到机器学习方向，参与训练 Composer 模型，研究模型行为与“性格”[3]。2026 年 8 月 14 日 Cursor 完成并入 SpaceX，他转到 SpaceXAI，工作是让 Grok “useful, tasteful, and safe” [4]。
- CS146S（The Modern Software Developer）由 Mihail Eric 讲授，Fall 2026 学期每周二、四上课，内容涵盖 MCP、agent skills、spec-driven development、loop engineering 和 software factory，并邀请业界讲者 [2]。
- Grok Bot 于 2026 年 8 月 11 日以 beta 形式发布。官方介绍：每个 bot 有自己的云端电脑，登录用户已有的工具，在后台把任务做完，只在需要批准时回来找人；可以多个 bot 并行，也可以放进群聊互相协作 [5]。

1.2　怎样读这份笔记

笔记按讲座的六个部分组织，最后一章是综合整理。每张截图的图注第二行给出画面时间和对应讲解区间（mm:ss），可以回到视频核对。紫色虚线框“资料补充”里的内容来自讲座以外的公开资料，方括号编号对应文末“参考资料”；其余内容只依据视频本身。引用的英文原话取自 X 平台提供的英文字幕。

两点说明：

- 录像右下角有讲者的画中画，部分幻灯片右下角的文字被遮挡。能找到无遮挡画面时用了无遮挡帧，否则在正文中补全被遮住的内容。
- 讲座里“bot”有两层意思：泛指这一类常驻智能体，也特指 Grok Bot 产品。笔记中泛指时写“常驻智能体”，特指时写 Grok Bot。

1.3　本章小结

- 这是一场面向工程学生的架构讲座，以 Grok Bot 为例，从“模型在循环中调用工具”这一最小结构讲到可以连续运行数周的产品。
- 主线依次是：发展脉络、模型能力、系统结构、harness、上下文工程、未来方向。

2　从终端里的循环到常驻智能体

2.1　起点：第一周写过的循环

讲座的出发点是学生在第一周写过的代码（00:20–00:48）。它只有几行：维护一个消息列表，把列表连同工具清单发给模型；模型如果请求调用工具，就执行工具、把结果追加到列表，再发一次；模型不再请求工具时，打印回复并退出。

```python
messages = [system_prompt, user_message]

while True:
    reply = llm(messages, tools=TOOLS)
    messages.append(reply)
    if not reply.tool_calls:
        print(reply.text)
        break
    for call in reply.tool_calls:
        messages.append(run_tool(call))
```

代码 1　第一周的最简 harness（据 00:46 幻灯片转录）

Lee 认为这个结构已经能解锁很多能力，但它有一个硬伤：进程跑在你的终端里，合上电脑，进程就停了（00:32–00:40）。这场讲座要回答的问题是：怎样在保留这个循环的前提下，让智能体在服务器上持续运行，并且成为一种不同的产品。

2.2　常驻智能体多了什么

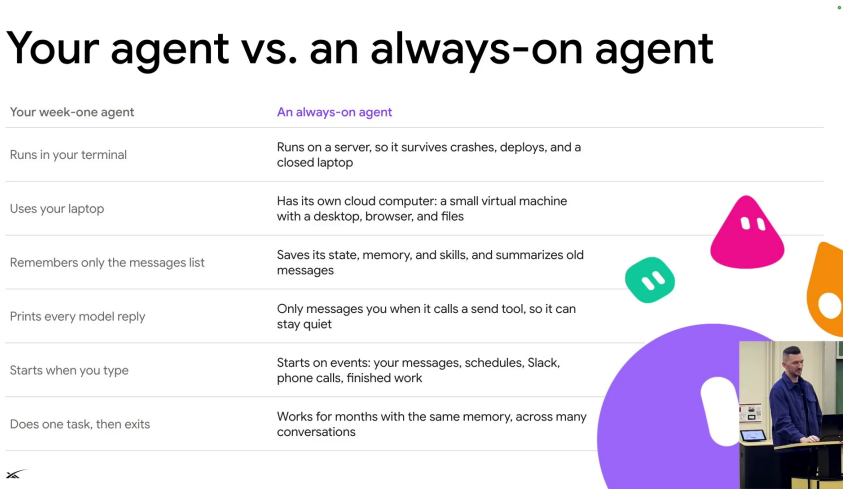


图 2.1　第一周的智能体与常驻智能体逐项对比
视频画面 02:06；讲解区间 00:48–02:08

图 2.1 把两者逐项对比。Lee 的讲解补充了几处细节（00:48–02:08）：

- **运行位置**：跑在服务器上，可以重启，可以扛过崩溃和部署。
- **自己的电脑**：不是借用你的笔记本，而是一台云端 Linux 机器，有桌面、文件、浏览器，必要时还能装 3D 软件。

- **记忆**：在机器上存文件，逐渐了解你的工作进度和习惯，旧消息会被摘要。
- **会保持安静**：终端里的 harness 每条消息都回复；常驻智能体只在调用“发送”工具时才给你发消息，可以先在后台干活，有重要进展再来找你。
- **触发方式**：不只是你打字，Slack 消息、邮件、定时任务、电话或其他事件都能把它唤醒。
- **时间尺度**：目标是在同一份记忆下工作数周甚至数月，跨越很多次对话。

> **要点｜常驻智能体的定义**
>
> 常驻智能体 = 第一周的“模型 + 工具循环” + 一台属于它的云端电脑 + 存在服务器上的状态与记忆 + 多种事件触发 + 自主决定何时发消息。它的行为更像同事：不是每条消息都立即回复，而是在后台工作，只在有重要更新时找你。

2.3 开发者与 AI 协作的四个阶段

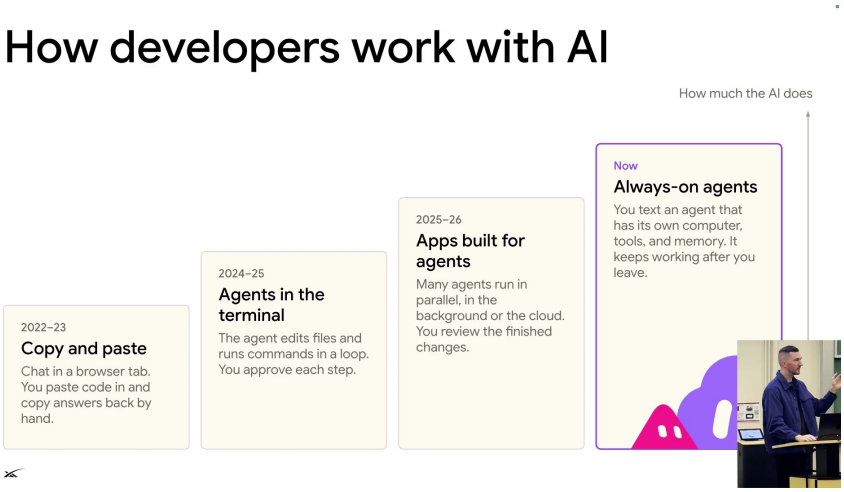


图 2.2　开发者使用 AI 的四个阶段，纵轴为“AI 承担的工作量”

视频画面 04:06；讲解区间 02:36–04:08

Lee 把过去几年分成四个阶段（02:36–04:08）：

1. **2022–23，复制粘贴**：在浏览器里和 ChatGPT 对话，手动把代码贴进编辑器。
2. **2024–25，终端里的智能体**：模型能编辑文件、执行 shell 命令，循环工作，但每一步都要人批准。
3. **2025–26，为智能体做的应用**：多个智能体并行，跑在后台或云端，人审阅最终改动。这一阶段人仍是瓶颈，因为要盯着每一条命令。
4. **现在，常驻智能体**：你给一个有自己电脑、工具和记忆的智能体发消息，你离开后它继续工作。

2.4 从逐步批准到随时纠偏

过去的流程是“提示—等待—批准—等待”。现在的流程是 steering：智能体持续工作，人随时补充指令。Lee 的例子是先说“去看一下这个 Slack 讨论串”，紧接着又说“对了，改到周四做”。上一代模型遇到这种连续打断会混乱，现在的模型和 harness 能理解两句话之间的关系，就像你跟朋友说话时前后补充一样（04:16–05:00）。

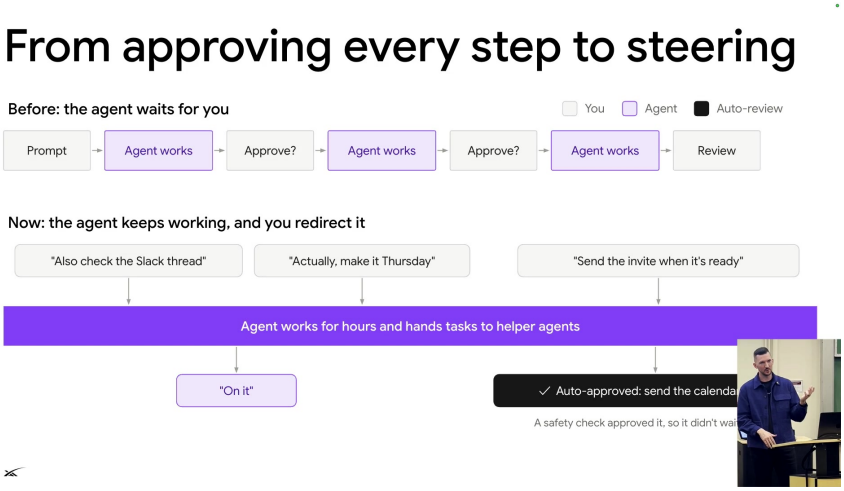


图 2.3　交互方式的变化：过去每一步等人批准，现在智能体持续工作，人随时插话纠偏，风险操作由自动审查放行

视频画面 05:24；讲解区间 04:08–05:28

发邮件、发 Slack 这类操作由一个独立的 AI 模型先审查，很多公司把这叫 auto-review。Lee 的判断是：比起让人在疲劳中逐条批准几百条 shell 命令，这种方式反而更安全；在盲测中，模型审查命令的表现多数时候好于人工（05:00–05:24）。这是讲者的经验判断，视频没有给出测试数据。

2.5　模型越强，界面越简单

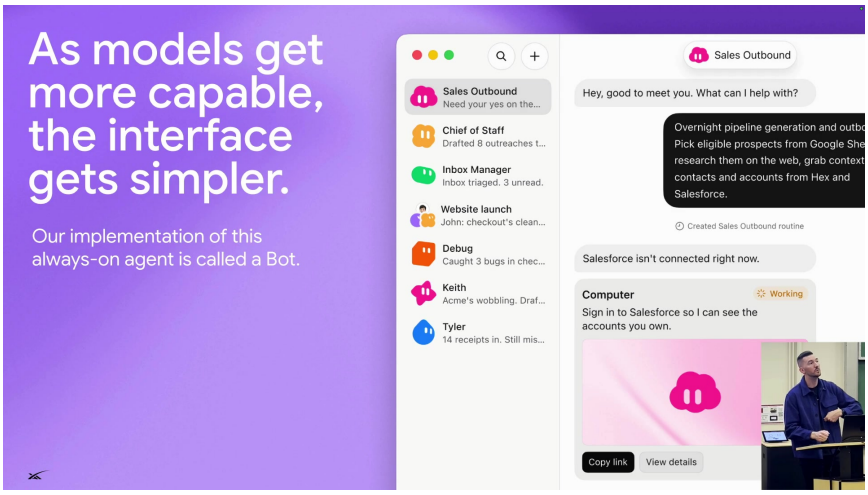


图 2.4　“模型能力越强，界面越简单”。左侧是多个 bot 的会话列表，右侧是一个销售外联 bot 在请求登录 Salesforce

视频画面 06:10；讲解区间 05:28–06:12

讲座把这种常驻智能体称为 bot（05:28），并指出同类产品正在大量出现。Lee 给出的原因是：人人都会发短信。把与 AI 的协作压缩成“给朋友发消息”的形式，而背后能完成真实的长任务，像交给同事一样（05:50–06:12）。图 2.4 里的会话列表就是这种形态：Sales Outbound、Chief of Staff、Inbox Manager、Debug 等，每个 bot 负责一类工作。

资料补充 | Grok Bot 的产品形态

- 账户下所有 bot 共用一台持久的云端电脑，共享其中的文件、浏览器会话和登录状态，方便交接；每个 bot 在这台电脑上有自己的屏幕，同一时间只运行一个 computer-use 任务 [6]。
- 关掉应用、合上笔记本或锁上手机，都不会停止正在进行的后台任务或定时例程 [6]。
- 官方介绍的“演示学习”：让 bot 跟着看你做一遍任务，它把流程存成 routine，接受修正，下次自己执行 [5]。

2.6 本章小结

- 第一周的循环能力很强，但进程依附于终端；常驻智能体把循环搬到服务器，再加上云端电脑、持久状态、事件触发和“默认安静”。
- 协作方式从“逐步批准”变成“随时纠偏”，风险操作交给独立的审查模型。
- 产品形态收敛到“发消息”，复杂度藏在后端。

3 模型发生了哪些变化

3.1 六项新能力

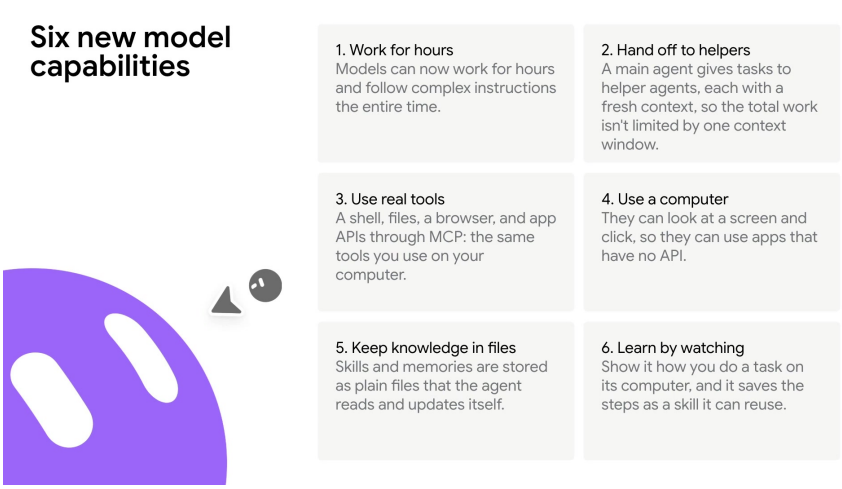


图 3.1　支撑常驻智能体的六项模型能力

视频画面 08:25；讲解区间 06:13–08:28

常驻智能体成立的前提是模型能力的变化。Lee 列了六项（06:13–08:28），并说实际不止这些：

1. **长时间工作**：过去一两年，模型在长时间内遵循复杂指令、不迷失的能力明显提升。配合压缩（compaction），现在可以较有把握地连续运行数小时甚至数天。
2. **交给助手**：主智能体把任务交给子智能体（sub-agent）。主智能体的工作记忆迟早会满，子智能体带着一份全新的上下文去做耗 token、耗工具调用的工作，再把结果报告回来，整体上等于获得了几乎不受限的上下文。
3. **使用真实工具**：shell、文件、浏览器，以及通过 MCP 接入的应用 API。Lee 提到，几年前模型几乎不会调用工具、经常幻觉，而现在已经很少有人谈幻觉问题。这是讲者的观察。
4. **使用电脑**：模型能看屏幕、点击，因此没有开放 API 或 MCP 服务器的 SaaS 和网站也能被自动操作，即 computer use。

5. **知识存在文件里**：技能和记忆都是纯文本文件，模型检索、读取和更新都很容易。
6. **看一遍就学会**：给它演示一遍，它把过程变成可复用的技能、记忆或例程。

资料补充｜MCP 是什么

Model Context Protocol（MCP）由 Anthropic 于 2024 年 11 月 25 日发布，是一个开源的开放标准，用来把 AI 应用连接到外部数据源和工具：MCP 服务器暴露数据和工具，MCP 客户端连接这些服务器，以一套协议替代各自定制的集成 [15]。第 5 章提到的“连接器”大多就是 MCP 服务器。

3.2 智能体知道的东西都放在文件里

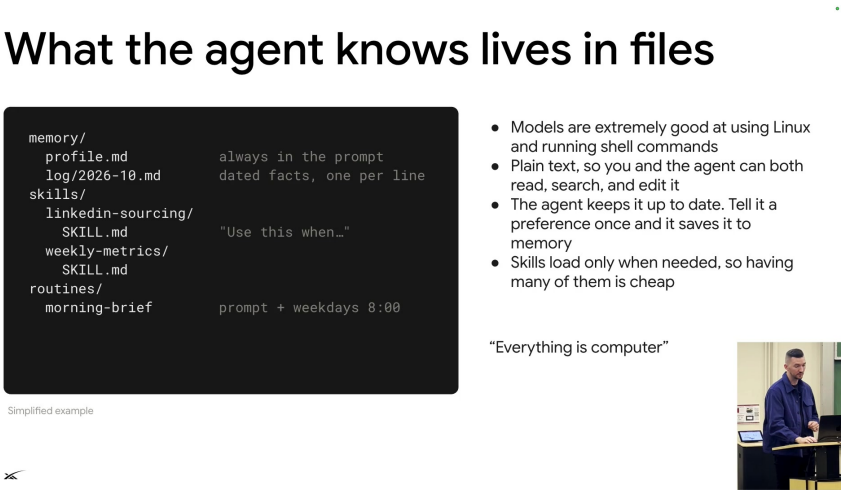


图 3.2　记忆、技能和例程都以文件形式存放（简化示例）

视频画面 10:02；讲解区间 08:28–10:04

图 3.2 是一个简化的目录结构（08:28–10:04）：

- `memory/profile.md`：用户画像，每次都放进提示词。Lee 的例子：如果你喜欢全小写的短信风格，这一点每条消息都要遵守。
- `memory/log/2026-10.md`：按日期记录的事实，一行一条，类似同事每天写的工作日志。问它“我上周做了什么”，它会去查这个文件。
- `skills/< 名称 >/SKILL.md`：技能文件，开头写明“什么时候用”。技能是把完成某项任务的具体流程、成败要点写成 Markdown。现在已有很多公司和开发者在写通用技能，例如怎样写好一封邮件、怎样设计 API。
- `routines/morning-brief`：例程，即“提示词 + 时间表”（例如工作日 8:00），相当于 cron 任务。

幻灯片右侧给出这样做的理由：模型非常擅长用 Linux 命令（grep 等）检索文件；纯文本让人和智能体都能读、搜、改；你说一次偏好，它自己写进记忆；技能只在需要时加载，所以技能多也不贵。Lee 还提到 Grok Bot 有一套插件系统，任何想用的工具都有对应插件，插件内部由技能或服务器组成（09:39–09:55）。这一部分的标语是“Everything is computer”。

资料补充｜Agent Skills 规范与“按需加载”

Agent Skills 规范规定：一个技能是一个目录，至少包含 `SKILL.md`，文件以 YAML frontmatter 开头，必填 `name`（不超过 64 个字符）和 `description`（不超过 1024 个字符，写明做什么、什么时

候用），可选 `scripts/`、`references/`、`assets/` 目录 [19]。规范描述的加载方式分三级：启动时只加载所有技能的 name 和 description（约 100 token）；技能被激活时加载 `SKILL.md` 正文（建议少于 5000 token）；脚本和参考文件在需要时才读取 [19]。这正是幻灯片所说“技能多也不贵”的机制。

3.3 本章小结

- 六项能力里，“长时间遵循指令”“子智能体”“computer use”分别解决了时长、上下文容量和覆盖面的问题。
- 把记忆、技能、例程都放进文件，是利用模型最擅长的操作（读写文件、跑命令）来管理长期知识。

4 常驻智能体的内部结构

4.1 没事的时候什么都不运行

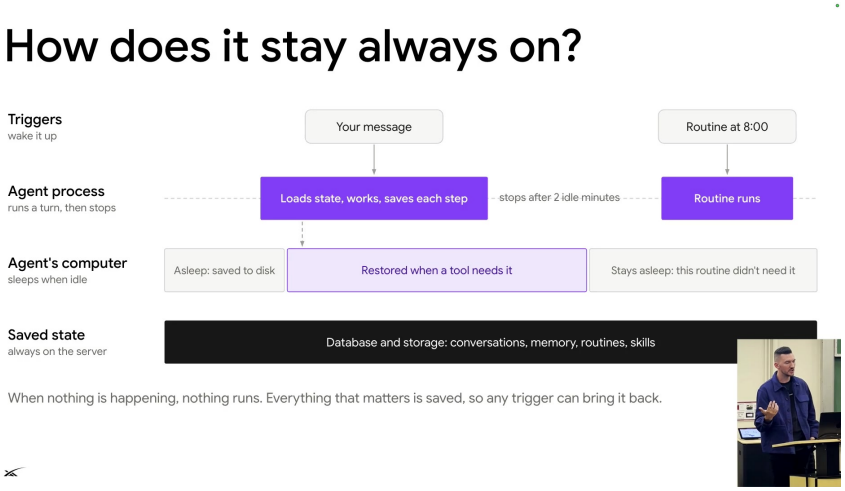


图 4.1　“常驻”的实现：触发时加载状态、跑完一轮就停；电脑只在工具需要时恢复；状态始终在服务器

视频画面 11:10；讲解区间 10:12–11:12

“常驻”并不是让进程一直跑。图 4.1 从左往右读（10:12–11:12）：

- 初始状态下，服务器上的智能体进程和它的电脑都在休眠。
- 你发消息或某个触发器触发时，进程启动，加载这个智能体的全部状态，干活，每一步都保存；空闲约 2 分钟后停止。
- 如果它需要用电脑（打开浏览器、点击界面），电脑才从磁盘恢复；不需要电脑的例程（如 8:00 的晨报）不会唤醒电脑。
- 会话记录、记忆、例程、技能都持久化在服务器的数据库和存储里。

这样设计的原因很直接：让每个用户的电脑一直开着，成本太高，Lee 的说法是“世界上也没有那么多电脑”，服务器开销也会很大（10:43–11:16）。幻灯片底部的总结是：没事发生时什么都不运行；重要的东西都已保存，任何触发都能把它唤回来。

4.2 什么会唤醒它

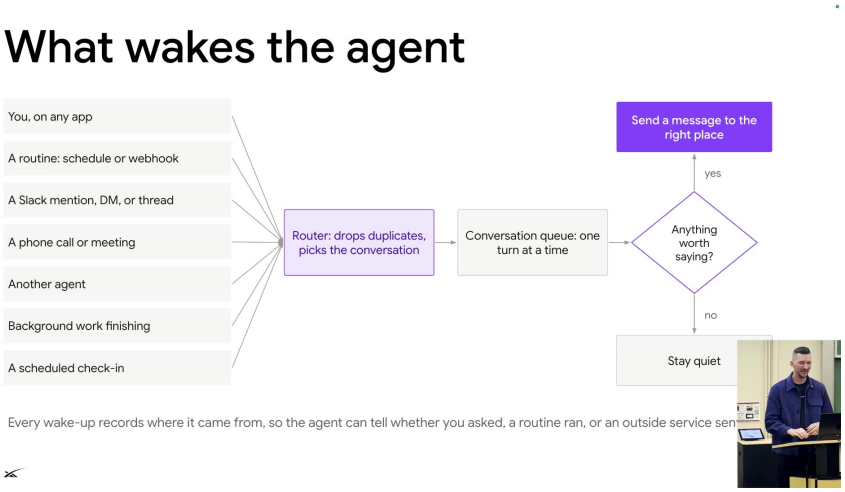


图 4.2　唤醒来源、路由、会话队列和“值不值得说”的判断

视频画面 12:18；讲解区间 11:12–12:20

唤醒来源有七类：任意应用里的你、例程（定时或 webhook）、Slack 提及/私信/讨论串、电话或会议、另一个智能体、后台工作完成、定时检查。Lee 提到，在 SpaceX 内部，员工赶不上会议时让自己的 bot 去开会、记笔记再回来汇报，已经越来越常见（11:30–11:45）。

所有事件先进入路由器：去重，并决定归入哪个会话；再进入会话队列，一次只处理一轮。每轮结束有一个判断：有没有值得说的？有，就把消息发到正确的地方；没有，就保持安静。Lee 用同事作比：如果同事每推进一点都来打扰你说“你看我又有进展了”，会很烦；多数任务每天一次状态更新就够了（11:52–12:22）。幻灯片还注明，每次唤醒都会记录来源，智能体因此能分辨是你提的要求、例程到点了，还是外部服务发来的事件。

4.3 智能体的电脑

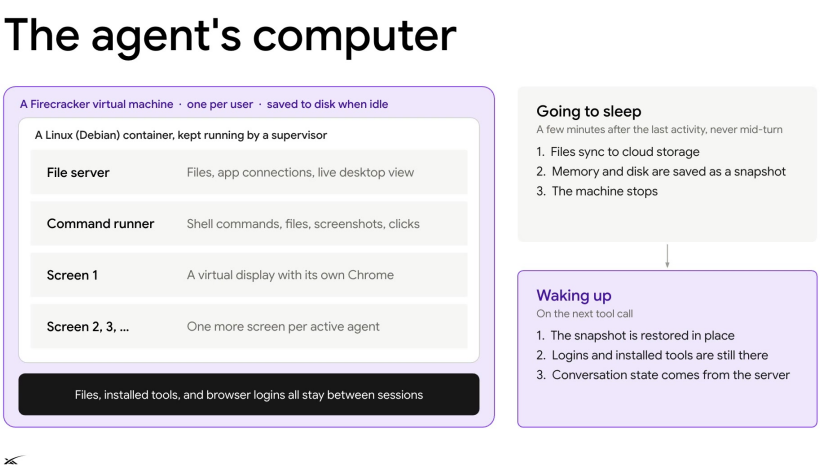


图 4.3　智能体的电脑：每个用户一台 Firecracker 虚拟机，空闲时存盘；右侧为休眠与唤醒的步骤

视频画面 12:29；讲解区间 12:20–13:12

这台电脑是 Firecracker 虚拟机，每个用户一台，空闲时保存到磁盘。选它的原因是很容易给整台机器打快照、关机、再恢复（12:23–12:40）。结构如下：

- 虚拟机里运行一个 Linux（Debian）容器，由 supervisor 保持运行。
- **文件服务器**：文件、应用连接、桌面实时画面。用户可以连进去看实时画面，也可以自己接管操作。
- **命令执行器**：shell 命令、文件、截图、点击。命令跑在一个隔离、锁定的环境里，而不是你的个人电脑。
- **屏幕 1、2、3……**：每个活跃的智能体一块虚拟屏幕，各自有 Chrome。同时跑五个、十个 bot 时，它们各用各的屏幕。
- 文件、安装的工具和浏览器登录状态在会话之间保留。

休眠与唤醒的顺序：

- **休眠**：最后一次活动几分钟后进行，绝不在一轮工作中途进行。先把文件同步到云存储，再把内存和磁盘存成快照，最后关机。
- **唤醒**：发生在下一次工具调用时。快照原地恢复，登录和已装工具都还在，会话状态从服务器取回。

资料补充｜Firecracker 与快照

- Firecracker 由 AWS 开发，最初用于 AWS Lambda，基于 KVM 在用户态创建 microVM；只模拟 5 种设备，启动时间低于 125 ms，每个 microVM 内存开销低于 5 MiB，单台主机每秒最多可创建 150 个 microVM [8]。
- 快照机制把运行中的 microVM 及其资源序列化保存，之后可在另一个 Firecracker 进程中恢复，客户机里的工作负载从那个时间点继续运行。加载快照只能在 microVM 启动前进行，必须提供完整内存快照和 VM 状态文件，原来的磁盘文件、TAP 网卡和 vsock 需在相同路径可用 [9]。
- 官方建议在与生成快照时相同的软硬件配置上恢复快照 [9]。

4.4 系统的三个部分

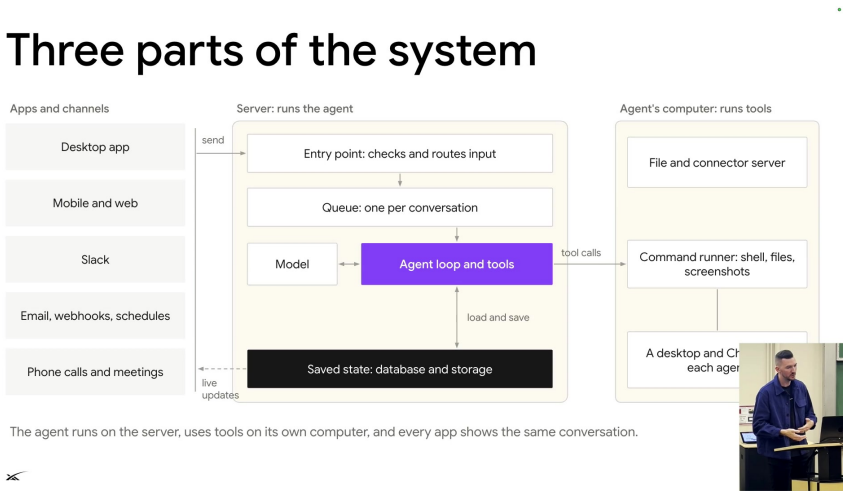


图 4.4　系统三部分：应用与渠道、运行智能体的服务器、运行工具的电脑

视频画面 14:02；讲解区间 13:12–14:04

把前面几块拼起来，就是图 4.4（13:12–14:04）：

- **左：应用与渠道**。桌面应用、移动端和网页、Slack、邮件/webhook/定时任务、电话和会议，都向服务器发送输入，并接收实时更新。
- **中：服务器**。入口检查并路由输入，每个会话一个队列，紫色方框是 agent loop（模型在循环中调用工具，即上一周的内容），状态存入数据库和存储。
- **右：电脑**。文件和连接器服务器、命令执行器（shell、文件、截图）、每个智能体一个桌面和 Chrome。

Lee 的评论是：整个产品的核心只是中间那个小小的循环，其余都是在它之上加的抽象，再接上有用的工具和外部服务（13:56–14:04）。幻灯片底部的总结：智能体运行在服务器上，用自己电脑上的工具，所有应用显示同一个会话。

4.5 为什么 agent loop 放在服务器上

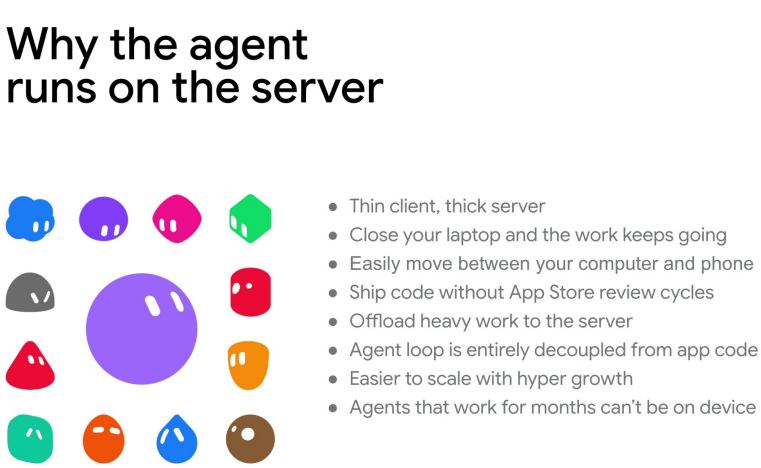


图 4.5　agent loop 放在服务器上的八条理由
视频画面 14:09；讲解区间 14:04–15:04

Lee 把这归为计算领域常见的“瘦客户端、胖服务器”取舍（14:00–15:04）。几条理由：

- 更新智能体代码只需部署服务器，不必等 App Store 审核。
- 转写、图片生成等重计算放在服务器上，通常比在设备上快。
- 服务器应对流量波动和高速增长有成熟办法。
- 能在电脑和手机之间无缝切换；工作几个月的智能体不可能放在设备上。

他强调最关键的一条：如果把 agent loop 放进虚拟机，就不能独立于智能体关停虚拟机。例如遇到紧急的 Linux 零日漏洞需要重建虚拟机，你希望重建电脑，而不是把整个 agent loop 一起毁掉、让应用停机（14:50–15:22）。所以紧接着一页幻灯片只有一句话：“Save everything on the server. The virtual machine can then always be rebuilt.”

> **要点｜状态与计算分离**
> 状态（会话、记忆、例程、技能）只存在服务器上；虚拟机是可以随时丢弃、重建的执行环境；agent loop 也在服务器上。三者解耦后，电脑可以休眠、打补丁、重建，智能体本身不受影响。

4.6 点击“发送”之后发生了什么

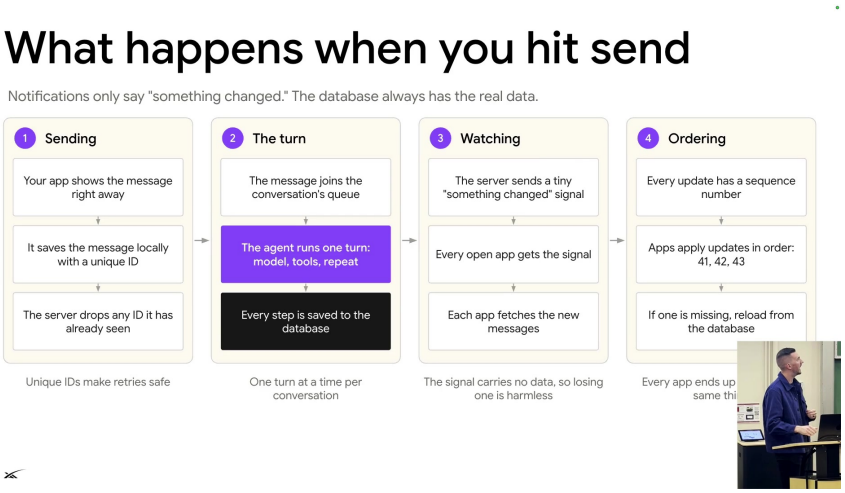


图 4.6　一条消息的四个阶段：发送、执行一轮、通知、排序

视频画面 16:14；讲解区间 15:24–16:16

Lee 说这部分偏细节，涉及强一致性与最终一致性等分布式系统话题（15:24–16:16）。幻灯片标题下的一句话是要点：通知只说“有东西变了”，真实数据永远在数据库里。四个阶段：

1. **发送**：应用立刻显示这条消息，并在本地用唯一 ID 保存；服务器丢弃已见过的 ID。唯一 ID 让重试变得安全，重试不会产生重复消息。
2. **执行一轮**：消息进入会话队列；智能体跑一轮“模型、工具、重复”；每一步都写入数据库。每个会话同一时间只跑一轮。
3. **通知**：服务器发一个很小的“有变化”信号，所有打开的应用收到后各自拉取新消息。信号本身不带数据，丢一个也无妨。
4. **排序**：每条更新带序号，应用按 41、42、43 的顺序应用；发现缺号就从数据库重新加载。最终每个应用显示相同的内容。

4.7 持久化工作流与任务队列

底层原语的选择是 workflow 还是 job queue（16:11–17:20）。设想智能体在执行一个多步骤任务，做到第 3 步时基础设施出了问题：

- **普通任务队列**：重启后从第 1 步重做，已完成的工作再做一遍，可能引发问题。
- **持久化工作流（durable workflow）**：每一步都有记录，新服务器从最后完成的那一步之后接着做，不重复劳动。

幻灯片下方三点：可以从任意位置恢复；步骤要写成“执行两次也无害”，即幂等（idempotent），这样重试才安全；计时器、取消、健康检查等能力由 Temporal 这类引擎内置。Lee 说他们使用开源的 Temporal，避免从零实现这些复杂机制，但基础设施仍需自己运维（16:58–17:20）。

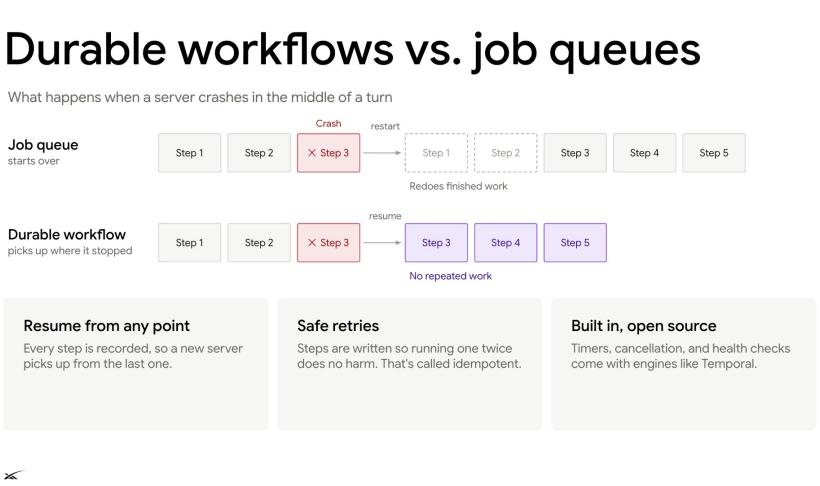


图 4.7　服务器在一轮中途崩溃时：任务队列从头重做，持久化工作流从断点继续

视频画面 17:09；讲解区间 16:16–17:20

资料补充｜Temporal 怎样做到“从断点继续”

- Temporal 为每次工作流执行维护一份按顺序记录的事件历史（Event History），它是工作流状态的唯一依据，持久化在 Temporal 服务的数据库里 [10]。
- 恢复时 Temporal 不是从内存快照还原，而是从头重新执行工作流代码，用事件历史引导代码回到之前的状态；已经完成的 Activity 和计时器直接返回历史里记录的结果，不会再次执行 [10, 11]。
- 因此工作流代码必须是确定性的；与外部世界交互的操作（调用 API、发消息等）放在 Activity 里 [10]。

注意｜“可重试”依赖幂等

持久化工作流保证的是“已完成的步骤不重做”。正在执行时崩溃的那一步仍可能被重试，所以幻灯片要求每一步“执行两次也无害”。第 4.6 节的唯一消息 ID 是同一思路：服务器丢弃见过的 ID，重试就不会产生重复消息。

4.8 新消息的处理规则

不同来源的消息优先级不同（17:20–18:08）：

- 你在一对一聊天里说“其实改到周四”或点“停止”，这条消息插到队首，打断或停止正在执行的一轮。
- 其他来源（群聊、Slack 讨论串、例程、助手完成通知）排到队尾。
- 短时间内连续收到三四条消息（例如“加上 Maya?”“还有 Jordan”“谢谢!”），合并成一批，只跑一轮。合并的启发式可以按需设计。

4.9 本章小结

- 常驻靠的是“随时能被唤醒”，不是一直运行：进程跑完一轮就停，电脑按需从快照恢复。
- 服务器保存全部状态，虚拟机可丢弃重建；agent loop 放在服务器上，客户端只负责显示。
- 消息链路的可靠性来自唯一 ID 去重、无数据的变更信号、序号排序和持久化工作流。

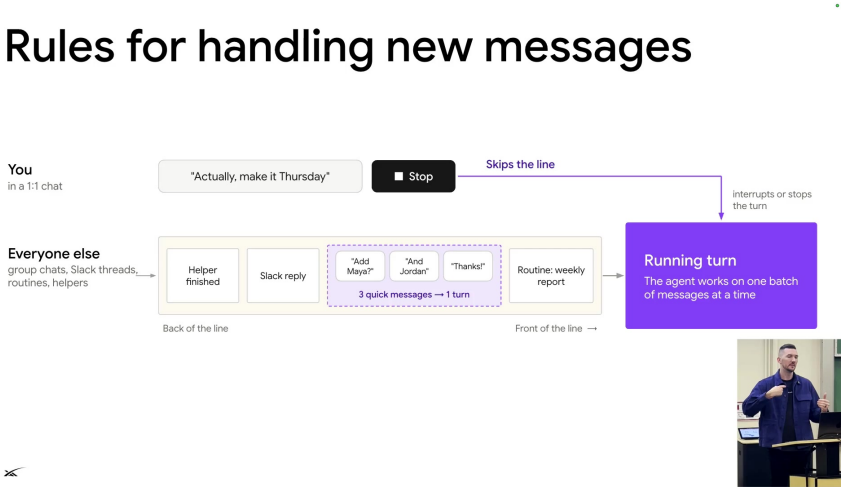


图 4.8　新消息的优先级：一对一聊天中的你可以插队、打断或停止当前一轮；其他来源排队，连续短消息合并为一轮

视频画面 18:06；讲解区间 17:20–18:08

- 用户的一对一消息优先级最高，其他事件排队，短消息批量处理。

5　Harness：工具、界面与安全

5.1　只有一个工具负责和你说话

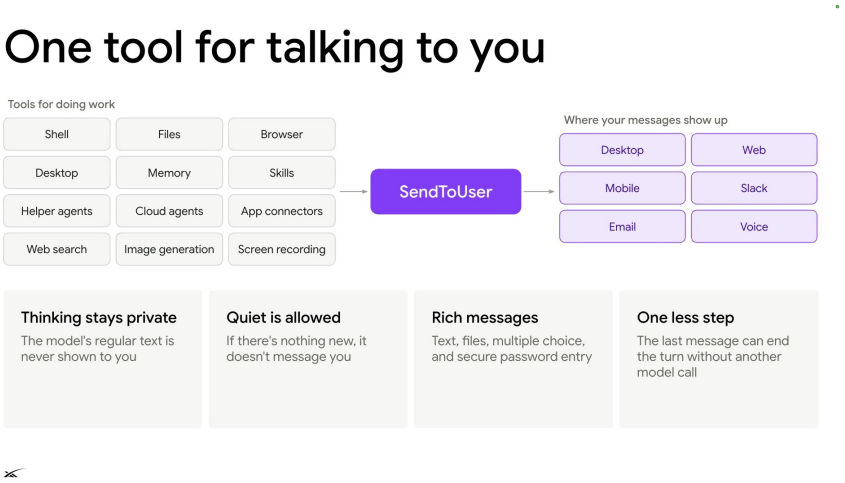


图 5.1　干活的工具有十几种，和用户说话的只有 SendToUser 一个；消息可以出现在桌面、网页、手机、Slack、邮件和语音

视频画面 19:14；讲解区间 18:20–19:16

Lee 认为 Grok Bot 的 harness 与市面上同类工具最大的不同在于：客户端和服务器之间只有一个通信工具，叫 SendToUser（18:20–19:24）。左边是干活用的工具：shell、文件、浏览器、桌面、记忆、技能、助手智能体、云端智能体、应用连接器、网页搜索、图片生成、录屏；它们都在服务器侧。右边是消息的去处：桌面、网页、移动端、Slack、邮件、语音。

架构因此很简单：客户端某种意义上只是在和服务器“发短信”。无论新接入的是移动应用、桌面应用、邮件还是 Slack，都只需要对接这一个工具。幻灯片下方列出四个性质：

- **思考不外露**：模型的普通文本输出不会显示给你，只有调用 SendToUser 的内容才会。
- **允许沉默**：没有新进展就不发消息。
- **富消息**：文本、文件、多选项、安全的密码输入。
- **少一步**：最后一条消息可以直接结束这一轮，不需要再调用一次模型。

5.2 工具配上专用界面

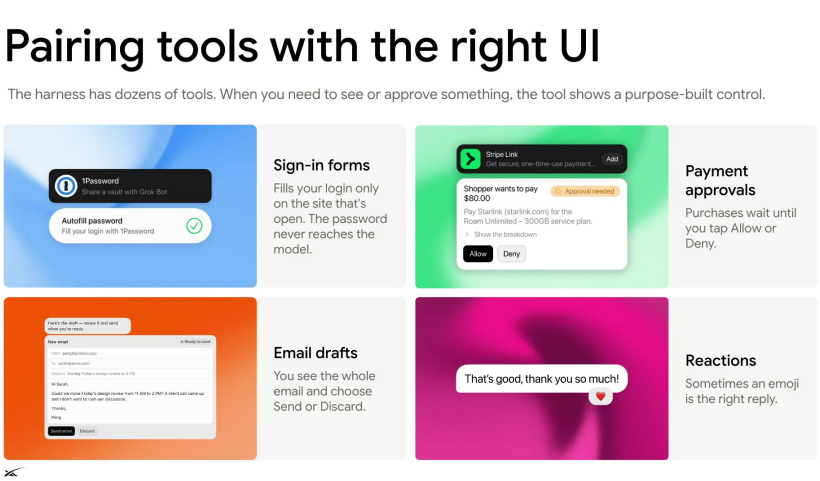


图 5.2　需要查看或批准时，工具显示专门的控件：登录表单、支付审批、邮件草稿、表情回应
视频画面 19:17；讲解区间 19:16–20:00

只有一个通信工具，但客户端可以根据收到的内容，用最合适的交互方式显示（19:16–20:00）：

- **登录**：显示专门的登录表单，可以对接 1Password，只在当前打开的网站填入登录信息，密码不经过模型。
- **支付**：对接 Stripe Link 等支付服务，购买操作等你点“允许”或“拒绝”。
- **不可撤回的操作**：发邮件、发 Slack 前，先给你一份可编辑的草稿，由你选择发送或丢弃。
- **表情回应**：不需要回复文字时，只回一个表情。

5.3 工具详情按需加载

每个模型的上下文窗口有 token 上限，Lee 称之为模型的“工作记忆”（19:58–21:32）。每个工具都带名称、schema 和各种说明。装上 Google Drive、Gmail、YouTube 等连接器后，每个 MCP 服务器又带来一批工具，上下文很快被占满（图 5.3 上排），留给真正对话的空间就少了。

很多主流 harness 的做法是动态上下文，也有人叫 tool search tool：每次请求只放工具名称，模型决定要用某个工具时，再从文件里读取它的详情。Lee 说这些东西“一路往下都是文件和文件系统”。这样做节省上下文，也就节省了成本和用量。

资料补充｜Anthropic 的 Tool Search Tool 数据

- Anthropic 举例：GitHub（35 个工具，约 26K token）、Slack（11 个，约 21K）、Sentry、Grafana、Splunk 五个服务器共 58 个工具，对话开始前就占用约 55K token；内部还见过工具定义在优化前占用 134K token [14]。
- 对比测试中，预先载入 50 多个 MCP 工具约需 72K token，开始工作前共占用约 77K；改用

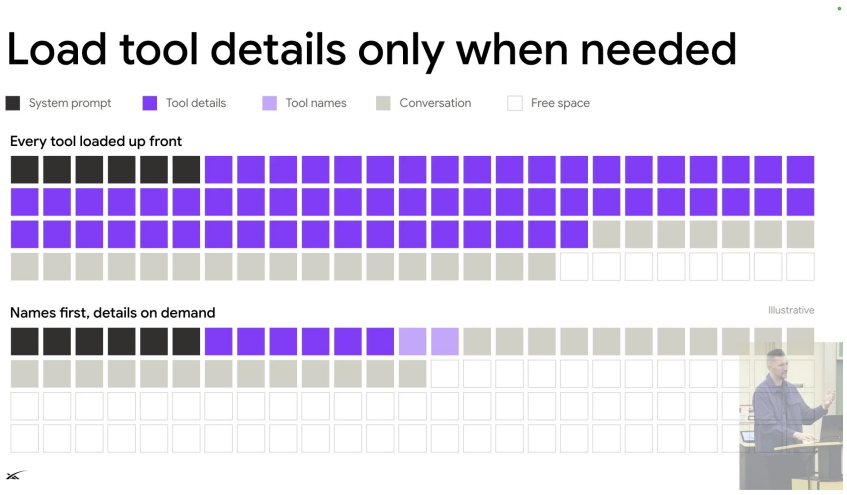


图 5.3　上下文窗口的占用示意：上排预先载入全部工具详情，下排只放工具名、详情按需读取

视频画面 21:30；讲解区间 20:00–21:32

Tool Search Tool 后，预先只载入搜索工具本身（约 500 token），按需载入 3–5 个相关工具（约 3K），总占用约 8.7K，减少约 85% [14]。

- 在大工具库的 MCP 评测中，启用后 Opus 4 的准确率从 49% 升到 74%，Opus 4.5 从 79.5% 升到 88.1%。被延迟加载的工具不在初始提示词里，因此不会破坏提示词缓存 [14]。

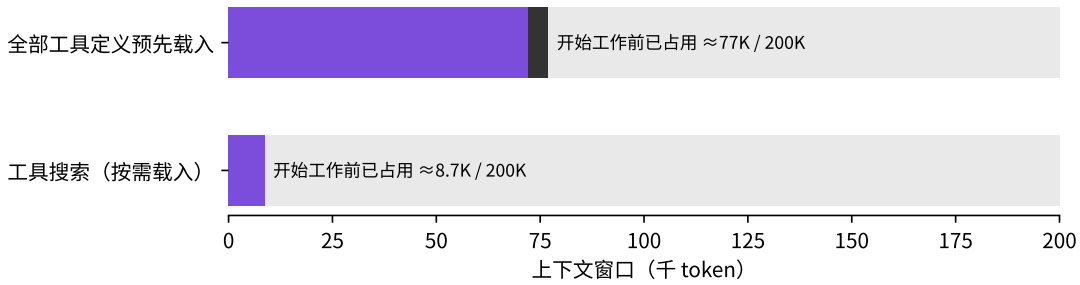


图 5.4　按 Anthropic 公布的测试数据绘制的上下文占用对比（窗口按 200K 计）。数据来源：[14]

5.4　先试最便宜的办法

原则是先试最便宜、最可靠的办法，再逐级往上（21:32–22:36）：

1. 它已经知道的：对话历史里就有。
2. 应用 API（连接器）：插件、MCP 服务器或 API，例如通过 Plaid 拉取银行信息。
3. 网页搜索：例如“昨晚小熊队比赛比分是多少”。
4. 它自己已登录的浏览器：在电脑上打开浏览器搜索、操作。
5. 完整桌面：跑脚本、跑命令、在 Linux 电脑上搭东西。
6. 问你：以上都不行，才来打扰你，请你提供输入或批准。

幻灯片顶部有一条限制：连接器坏了，智能体不应该用浏览器绕过去，而应该告诉你并询问。Lee 的类比还是同事：好同事会先自己试各种办法，而不是事事来问。

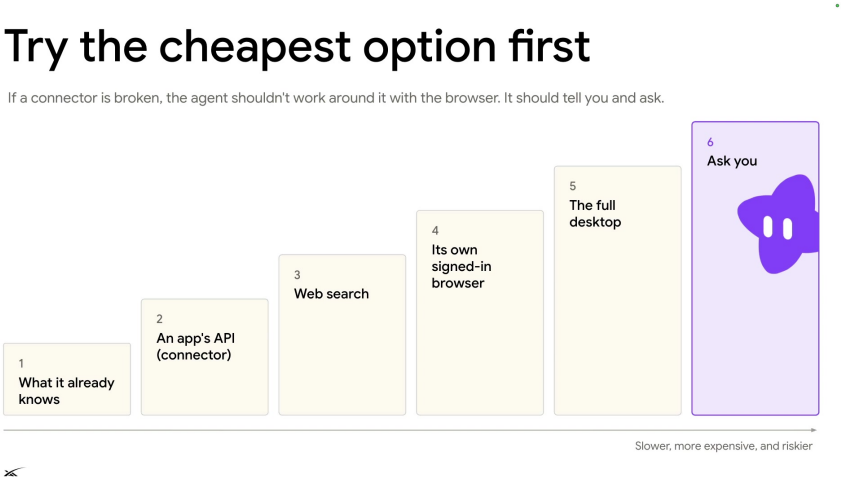


图 5.5　获取信息的六级阶梯，越往右越慢、越贵；连接器坏了不应该绕道用浏览器，而应告诉用户并询问

视频画面 21:37；讲解区间 21:32–22:36

5.5 使用电脑的三种方式

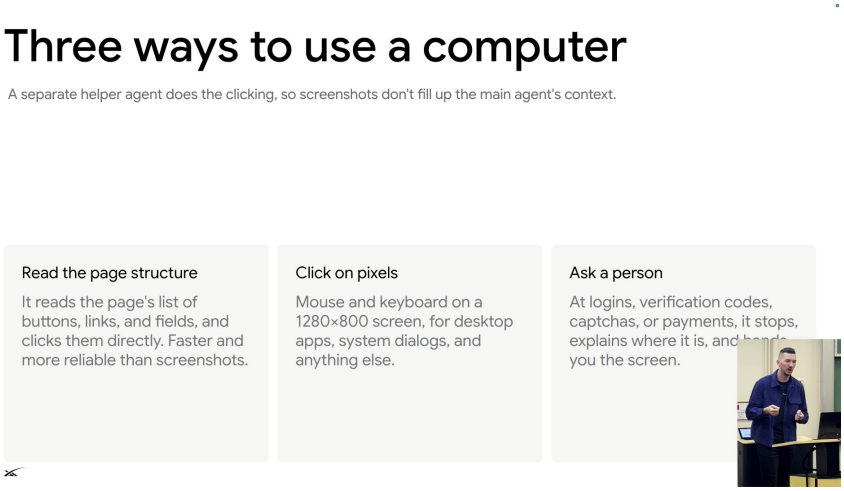


图 5.6　使用电脑的三种方式：读页面结构、点像素、交给人；点击由单独的助手完成，截图不进主智能体上下文

视频画面 23:26；讲解区间 22:36–23:28

在那台 Linux 电脑上获取页面信息，可以截图，也可以读 HTML，但更省的办法是读可访问性树（accessibility tree）：它能大幅减少 token，并直接给出可点击的按钮、链接和输入框（22:33–23:07）。三种方式：

- **读页面结构**：读取页面上按钮、链接、字段的列表，直接点击。比截图更快、更可靠。
- **点像素**：在 1280×800 的屏幕上用鼠标和键盘操作，用于桌面应用、系统弹窗和其他情况。有些网站会弹出必须点掉的对话框。
- **交给人**：遇到登录、验证码、CAPTCHA 或支付时停下，说明当前位置，把屏幕交给你。

幻灯片副标题说明了结构上的安排：点击由一个单独的助手智能体完成，截图因此不会塞满主智能体的上下文。

资料补充｜可访问性快照：Playwright MCP 的做法

Playwright MCP 默认用可访问性快照代替截图：每次交互后返回页面可访问元素的结构化树，每个元素带一个引用编号（如 `ref=e5`），点击、输入时直接传这个编号。官方对比：快照只有文本，token 成本低、可精确定位、结果确定；截图需要视觉模型和坐标猜测，布局一变就可能失效。画布、图表这类页面才需要配合截图使用 [16]。

5.6 一个主智能体，多个助手

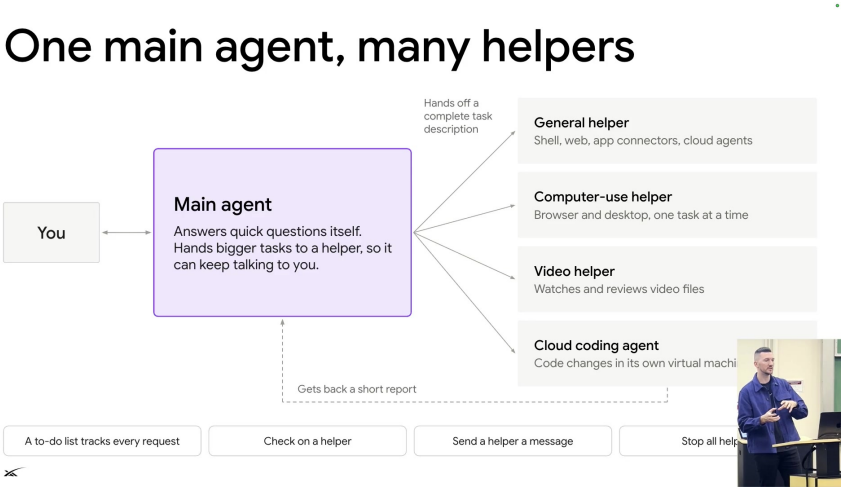


图 5.7　主智能体与四类助手：通用助手、computer-use 助手、视频助手、云端编码智能体；助手带完整任务描述出发，只带回简短报告

视频画面 24:36；讲解区间 23:28–25:12

主智能体是编排者（23:28–25:12）。简单问题它自己回答，大任务交给助手，自己继续和你对话。助手有四类：通用助手（shell、网页、应用连接器、云端智能体）、computer-use 助手（浏览器和桌面，一次一个任务）、视频助手（观看和审阅视频文件）、云端编码智能体（在自己的虚拟机里改代码）。主智能体用一份待办列表跟踪每个请求，可以查看某个助手的进度、给它发消息，或停止所有助手。

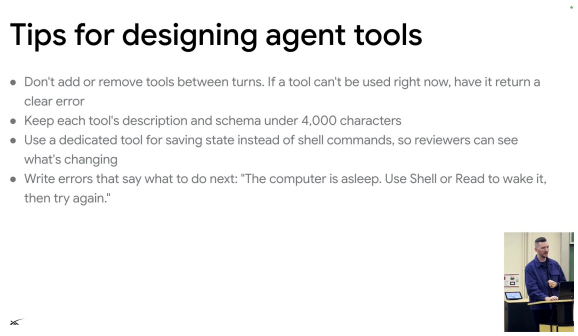
关键在于：助手执行大量 shell 命令或昂贵的工具调用，这些过程都不进入主智能体的上下文，回来的只有结果。Lee 说，这就是让产品感觉像有“无限上下文”的办法；无限上下文当然不存在，但配合这些技巧、并把模型训练得擅长这样做之后，用户不会感到撞上上下文上限，也不会觉得对话越长质量越差（23:41–24:44）。每个助手本身也是一个持久化工作流，比如视频助手内存耗尽崩溃，重启后接着做，不丢进度。

资料补充｜多智能体架构的收益与代价

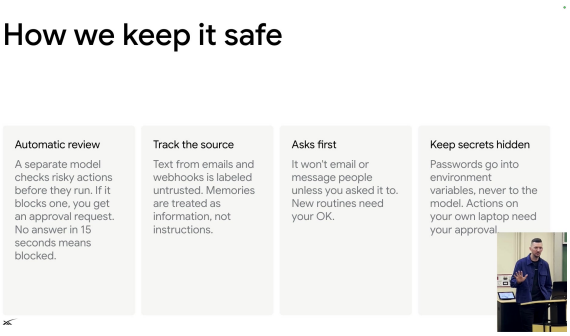
- Anthropic 的 Research 功能采用编排者—工作者模式。在其内部研究评测中，以 Claude Opus 4 为主智能体、Claude Sonnet 4 为子智能体的多智能体系统，比单智能体 Opus 4 高 90.2% [18]。
- 在 BrowseComp 评测中，token 用量单独就解释了 80% 的表现差异；代价是智能体通常比聊天多用约 4 倍 token，多智能体系统约 15 倍 [18]。
- 子智能体可能用掉数万 token 探索，但只返回 1,000–2,000 token 的浓缩摘要，细节留在子智能体的上下文里 [17]。

5.7 设计工具的几条经验

团队在实践中得到的几条经验（25:12–26:28）：



(a) 设计智能体工具的四条经验



(b) 安全机制的四个方面

图 5.8　工具设计与安全机制

视频画面 26:26 / 27:22；讲解区间 25:12–26:28 / 26:28–27:24

- 不要在两轮之间增删工具（会破坏提示词缓存，见第 6 章）。某个工具暂时不能用，就让它返回清楚的错误。
- 每个工具的描述和 schema 控制在 4,000 字符以内。
- 保存状态用专门的工具，不要用 shell 命令，这样审阅者能看清改了什么。Lee 补充：有人说 harness 只需要一个 shell 工具，这确实可行；但如果模型 90% 的时间都在用 shell 做同一件事，为它做一个专用工具更高效，人审阅时也更清楚。
- 错误信息要说明下一步怎么做。幻灯片的例子："The computer is asleep. Use Shell or Read to wake it, then try again."给人写的好错误信息，对智能体同样有用。

5.8　安全机制

让 bot 长期替你工作，前提是信任它对信息和操作的处理。即使它在自己的机器上运行、不污染你的电脑，也仍然需要多层保护（26:28–27:24）。右图的四个方面：

- **自动审查**：一个独立的模型在风险操作执行前检查；被拦下时你会收到批准请求，15 秒内无响应即视为拦截。
- **追踪来源**：来自邮件和 webhook 的文本标记为不可信；记忆被当作信息而不是指令。Lee 举例：收件箱里的邮件可能是钓鱼邮件，不能把它当成用户指令交给模型。
- **先问再做**：除非你要求，它不会给别人发邮件或消息；新建例程需要你同意。
- **密钥不进模型**：密码放进环境变量，不交给模型；在你自己笔记本上执行的操作需要你批准。

资料补充 | Grok Bot 安全文档的补充

- Auto Review 是一个独立的审查模型，覆盖 shell 命令、插件调用、computer use、自动化写入（例程和事件触发器的改动）以及 Cloud Agent、子智能体等委派操作，结论可以是放行、要求批准或拒绝 [7]。
- 它不审查所有副作用，例如记忆写入和多数设置改动；文档建议把它和不依赖模型判断的控制手段一起用：逐项批准、网络策略、按用户隔离 [7]。
- 外部内容（网页、插件结果、命令输出）在交给模型时标记为不可信数据；文档明确说这些措施降低而不是消除提示词注入风险 [7]。
- bot 没有自己的身份或凭据，以登录成员的身份行事；连接器的令牌保留在后端，不存放在电脑上 [7]。

> **注意｜提示词注入仍未解决**
>
> Lee 在讲座中说，确保不可信输入不被当作指令，“业界还有很多工作要做”（26:56–27:10）；第 7 章的开放问题里，他也把提示词注入列为“防御仍处于早期”。

5.9 本章小结

- 干活的工具在服务器侧，和用户说话的只有 SendToUser；客户端负责把它渲染成登录框、审批卡、草稿或表情。
- 工具详情按需加载，获取信息从最便宜的办法开始，截图和点击交给助手。
- 主智能体只编排，重活交给持久化的助手，回来的只有报告。
- 安全靠多层叠加：审查模型、来源标记、先问再做、密钥隔离。

6 上下文工程

6.1 目标：让上下文尽量少

Lee 说上下文工程（context engineering）也可以叫 harness 工程，两个说法随时间演变，基本是一回事：用各种技巧减少 harness 里的上下文用量，以提高效率、降低成本（27:26–27:56）。本章的五页幻灯片分别讲缓存、上下文瘦身、压缩时机和记忆分层。

6.2 提示词开头保持不变

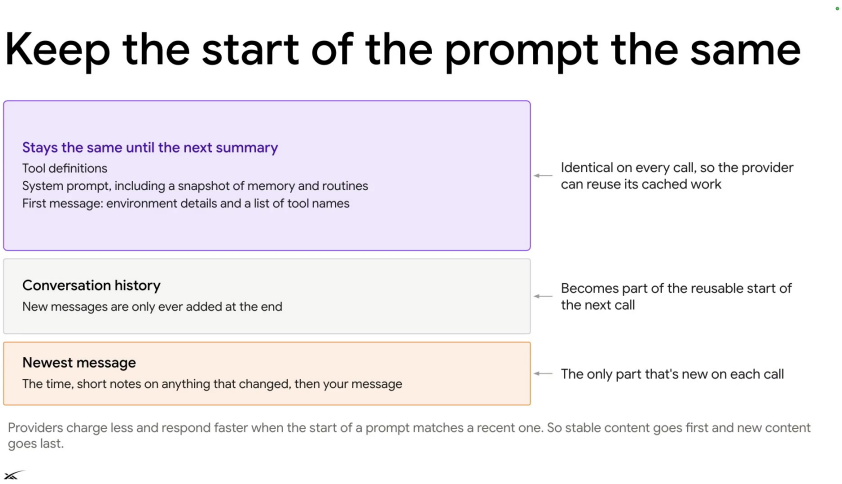


图 6.1　提示词的三段结构：开头部分在下一次摘要前保持不变，历史只在末尾追加，只有最新一条是新的

视频画面 28:37；讲解区间 27:48–28:44

和编码智能体或任何智能体交互时，每加一条消息，整个对话都会重新发送一遍，所以理想情况下大部分 token 都应该是缓存命中的 token。模型定价对缓存 token 有很大折扣，要尽量让它们留在缓存里（27:56–28:29）。做法是把提示词分三段：

- **开头（下次摘要前不变）**：工具定义；系统提示词，其中包含记忆和例程的快照；第一条消息，即环境信息和工具名列表。每次调用都完全相同，服务商可以复用缓存。
- **对话历史**：新消息只追加在末尾，这一段会成为下一次调用可复用的开头。

- **最新消息**：当前时间、自上次以来变化的简短说明、你的消息。这是每次调用唯一的新内容，只有这部分按未缓存价格计费。

资料补充 | 提示词缓存的规则与价格

- Anthropic 的缓存按完整前缀匹配，顺序是 `tools`、`system`、`messages`；某一层的改动会让这一层及之后的缓存全部失效，例如修改工具定义会让整个缓存失效。命中要求缓存部分 100% 相同 [12]。
- 默认缓存有效期 5 分钟，每次命中免费续期；也可选 1 小时有效期。价格倍率：5 分钟缓存写入为基础输入价的 1.25 倍，1 小时缓存写入为 2 倍，缓存读取为 0.1 倍（部分新模型为 0.05 或 0.025 倍）[12]。
- OpenAI 的提示词缓存对符合条件的请求自动生效，可复用前缀至少 1,024 个 token；官方建议把稳定的指令、示例、工具定义放在前面，时间戳和用户相关数据放在后面 [13]。

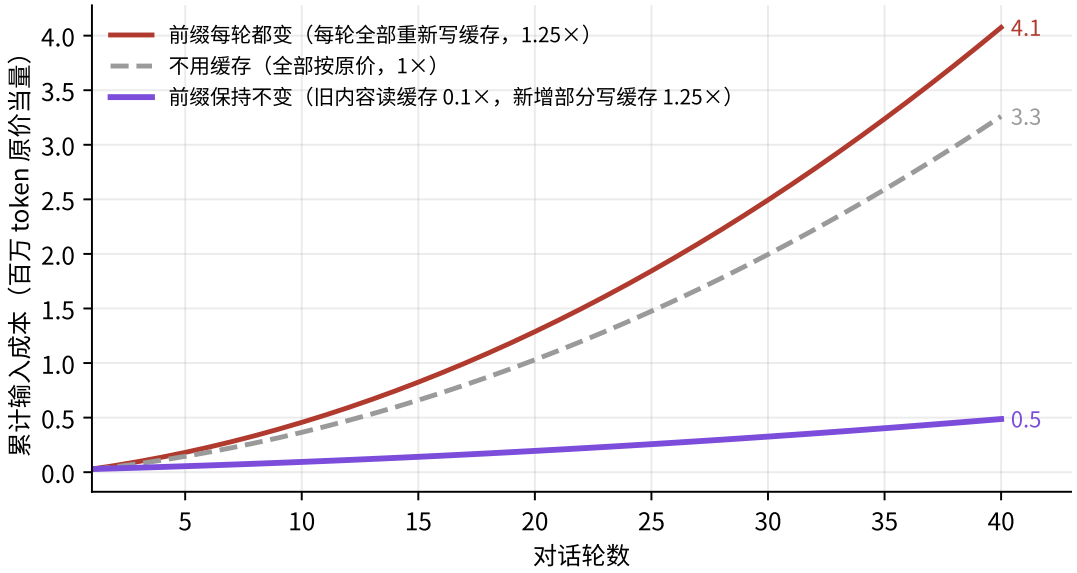


图 6.2　前缀稳定与否对累计输入成本的影响（示意）。假设固定前缀 2 万 token、每轮新增 3 千 token，共 40 轮；价格倍率取 Anthropic 文档的 5 分钟缓存：写入 1.25×、读取 0.1× [12]。前缀每轮都变时，每轮都要整段重新写缓存，比不用缓存还贵

6.3 缓存未命中当作 bug 处理

Lee 说，如果出现缓存未命中，那基本就是系统里的一个 bug（29:39–31:32）。幻灯片的六条做法：

- 在下一次摘要之前，工具、系统提示词和第一条消息必须一字不差。
- 如果一次部署改了某个工具的描述，正在进行的会话收到一条说明，而不是直接换成新文本。
- 所有内容按固定顺序构建并排序，调用之间不挪动位置。
- 不为某一轮单独增删工具，不可用的工具返回错误。
- 每一段分别计算哈希，可以准确看出是哪一部分变了。Lee 举例：做 A/B 测试、往提示词里插入新内容时，不能打乱所有人的提示词顺序，否则缓存就全失效了。



图 6.3　“把缓存未命中当作 bug”：六条具体做法
视频画面 29:53；讲解区间 29:52–31:32

- 你打开聊天或开始打字时，服务器就提前准备好提示词来预热缓存，类似网页在你悬停链接时预取下一页。

他还回忆了今年 1 月 OpenClaw 走红时的情况：很多人在模型厂商的订阅套餐里用它，这是厂商从未见过的使用模式，harness 也没针对这种工作流优化，于是出现大量缓存未命中；智能体全天运行时，这显然是问题，后来做了很多修复（29:56–30:42）。

资料补充｜OpenClaw 与缓存：几个数据点

- 德国科技媒体 c’t 3003 在 2026 年 1 月测试，在 Claude Opus 上运行 OpenClaw 一天消耗了 109.55 美元的 token；Anthropic 公布的 Claude Code 专业开发者日均成本约为 6 美元 [20]。
- 2026 年 4 月 4 日起，Anthropic 不再让 OpenClaw 等第三方 harness 使用订阅额度，改为计入额外用量。Claude Code 负责人 Boris Cherny 表示，他曾亲自给 OpenClaw 提交 PR 来提高提示词缓存效率 [20]。
- 一位开发者表示，把缓存 TTL 设为 1 小时、每 55 分钟发一次心跳之后，他的 token 消耗明显下降 [20]。
- 预热缓存在 API 层面也有直接支持：Anthropic 允许发送 `max_tokens: 0` 的请求，只写入缓存、不生成输出 [12]。

6.4　保持主上下文精简

主智能体是你与之进行超长对话的那个，它的上下文会在你不知不觉中被压缩很多次，所以必须保持精简（28:44–29:52）。右下角被画中画挡住的最后一条是 “Instructions become skills—The prompt keeps a one-line pointer”。六种办法：

- 很长的输出写进文件，上下文只留文件路径；智能体很擅长读文件，就像读技能或记忆一样。
- 截图交给助手，上下文只拿到助手的发现。
- 工具调用密集的工作交给助手，上下文只拿到简短报告。
- 旧消息被摘要，上下文只留简短摘要。
- 旧记忆保持可检索，提示词里只放最重要的事实。

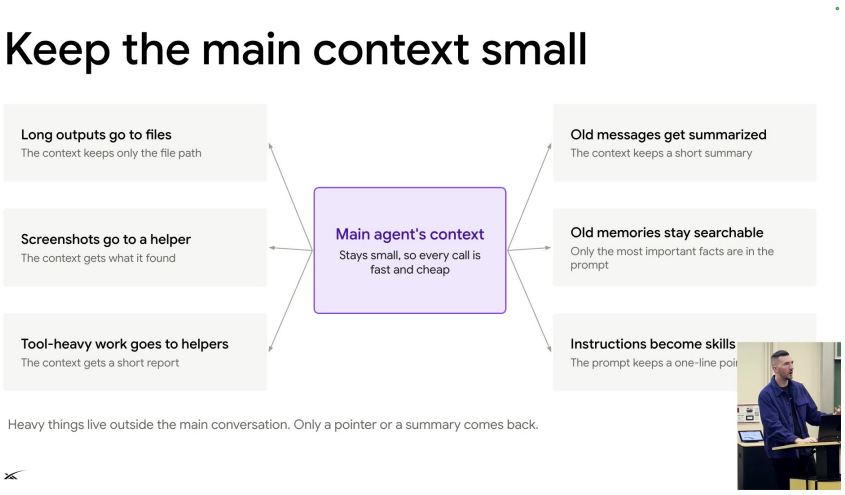


图 6.4　保持主智能体上下文精简的六种办法：重的东西放在主对话之外，只带回指针或摘要

视频画面 29:50；讲解区间 28:44–29:52

- 指令变成技能，提示词里只留一行指针。

Lee 的说法是要一边工作一边“修剪花园”，持续清理不必要的内容，既为成本，也为召回质量和模型表现（29:39–29:52）。

资料补充｜为什么上下文越长越差

- Anthropic 把这种现象称为 context rot：上下文中的 token 越多，模型准确回忆其中信息的能力越弱。原因之一是 Transformer 中每个 token 都要关注其他所有 token，n 个 token 有 n^2 对关系，上下文越长注意力越分散 [17]。
- 对应的做法是“即时”加载：只保留文件路径、查询语句、链接等轻量标识，需要时再用工具读入 [17]。
- 对压缩，Anthropic 建议先在复杂的智能体轨迹上把压缩提示词调到高召回，保证关键信息不丢，再逐步提高精确度、去掉多余内容；最安全、最轻量的一种压缩是清除早先的工具调用结果 [17]。

6.5　趁你离开时做压缩

长时间工作的关键是压缩（compaction），也就是摘要。假设模型有 20 万 token 的上下文窗口，一直聊下去，总会到需要摘要的时候；而所有压缩都是有损的，难点在于找到不丢重要细节的压缩算法（31:32–32:00）。

Lee 介绍了他们认为很有用的一个技巧（32:00–32:49）：你和 bot 来回聊了 10 万多 token，然后去做别的事。模型服务商的缓存有一个保温期，可能是 10 分钟，也可能是 60 分钟。趁缓存还热，立刻做压缩，这时读这段长对话很便宜；等你 30 分钟后回来，就从摘要开始，继续加消息也便宜得多。幻灯片下方三条细节：如果后台任务马上就要唤醒智能体，就跳过这次压缩；智能体最近发给你的几条消息逐字保留；如果某一轮本身太长，也可以在中途压缩。

6.6　记忆分层

压缩时选择把什么写进记忆，Lee 分了几层（32:52–34:28）：

- **画像（Profile）**：关于你的核心事实，每次都放进提示词，最多 100 条、4,000 字符。例如你一直要求全小写，那就必须每条消息都带上，否则突然冒出大写字母会很奇怪。

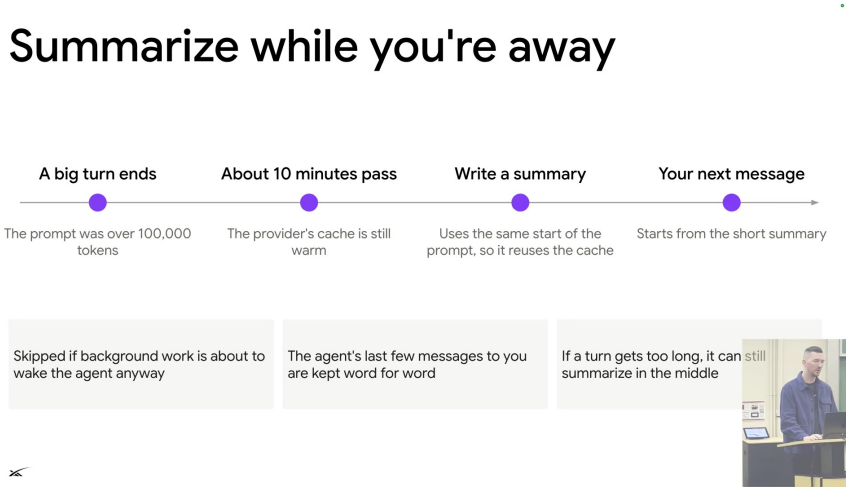


图 6.5　在缓存还热的窗口里写摘要：大轮次结束、约 10 分钟后、写摘要、你的下一条消息从摘要开始

视频画面 32:49；讲解区间 31:32–32:52

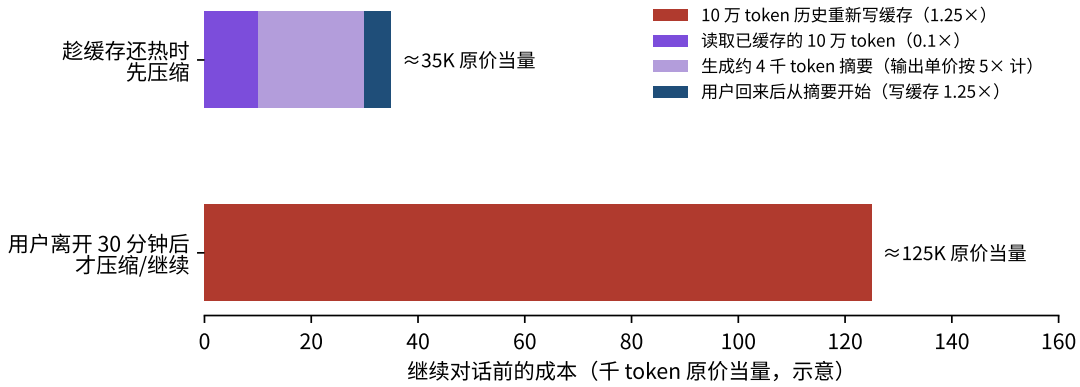


图 6.6　压缩时机的成本对比（示意）。假设对话 10 万 token、摘要 4 千 token；缓存写入 1.25×、读取 0.1× 取自 Anthropic 文档 [12]，输出单价按输入的 5 倍计（与该文档价格表中多数模型的输入/输出比一致）

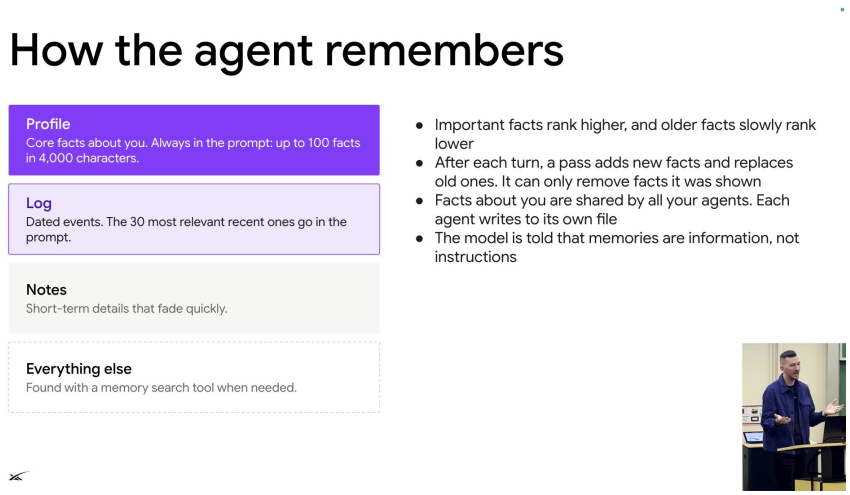


图 6.7　记忆的四层：画像、日志、笔记、其余一切；右侧为排序、更新和共享规则

视频画面 34:25；讲解区间 32:52–34:28

- **日志（Log）**：带日期的事件，最相关的 30 条近期记录进入提示词。
- **笔记（Notes）**：短期细节，很快淡出，可能只在接下来半小时到一小时有用。
- **其余一切**：需要时用记忆检索工具查找。

幻灯片右侧的规则：重要事实排序靠前，旧事实逐渐靠后；每轮结束后，有一个流程添加新事实、替换旧事实，但它只能删除自己看到过的事实；关于你的事实由你所有的智能体共享，每个智能体写自己的文件；模型被告知记忆是信息，不是指令。Lee 用人脑作比：此刻相关的事实留在脑中，与当下无关的（比如一场足球比赛）暂时不想，这像一种对事实的垃圾回收。

6.7 本章小结

- 缓存是长时间运行的经济基础：开头不变、历史只追加、新内容放末尾，任何未命中都按 bug 排查。
- 主上下文只留指针和摘要，重的内容放到文件和助手里。
- 压缩不可避免且有损；趁缓存还热时做压缩，成本最低。
- 记忆按“每次都要带”到“需要时再查”分四层，并有排序、替换和共享规则。

7 未来方向

7.1 为产品训练模型

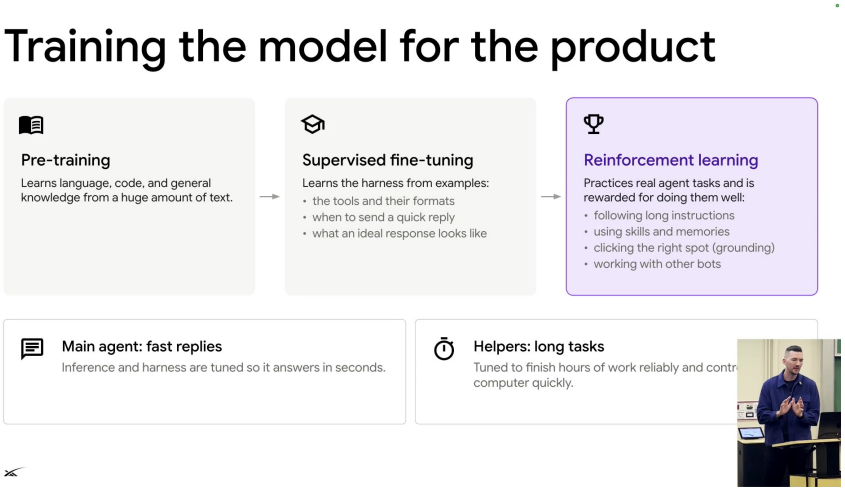


图 7.1　训练的三个阶段，以及主智能体与助手在推理侧的不同取向

视频画面 36:02；讲解区间 34:28–36:04

Lee 先回顾了大语言模型训练的三个阶段（34:28–36:04）：

- **预训练**：从海量文本中学习语言、代码和通用知识。
- **监督微调（SFT）**：从示例中学习 harness，包括有哪些工具、调用格式、什么时候该快速回复、理想的回复是什么样。
- **强化学习（RL）**：练习真实的智能体任务，做得好就得到奖励：遵循长指令、正确使用技能和记忆、点中正确的位置（grounding）、与其他 bot 协作。

推理侧也可以分开调：主智能体要低延迟、几秒内回复；交给助手的长任务，10 分钟还是 12 分钟差别不大，可以为可靠完成数小时工作、快速操控电脑去调。

7.2 产品与模型一起改进

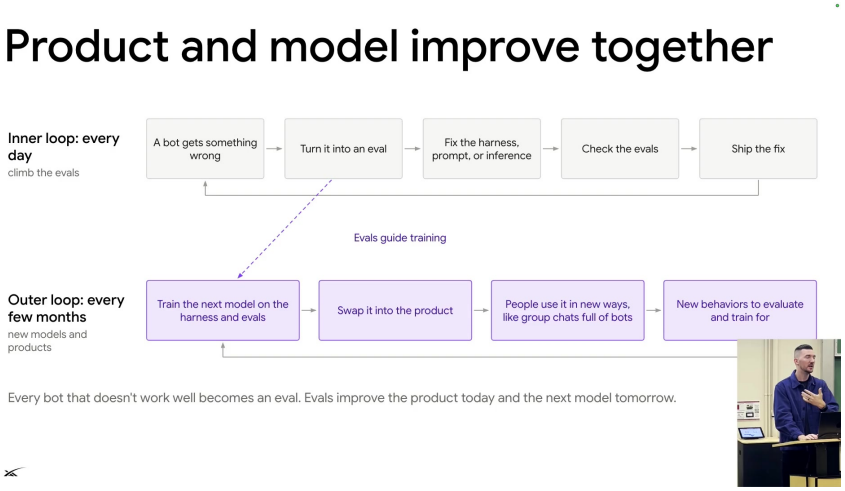


图 7.2　内循环每天改进 harness 与评测，外循环每隔几个月训练新模型

视频画面 37:42；讲解区间 36:04–37:44

图 7.2 有两个循环（36:04–37:44）：

- **内循环（每天）**：bot 做错了一件事，变成一条评测（eval）；在后端修 harness、提示词或推理问题；用评测确认已修好；发布修复。产品团队每天都在做这件事。
- **外循环（每几个月）**：用 harness 和评测训练下一代模型；换进产品；用户用出新方式，例如满是 bot 的群聊；产生新的待评测、待训练行为。

幻灯片的总结是：每个表现不好的 bot 都变成一条评测，评测既改进今天的产品，也改进明天的模型。Lee 特别提到，bot 在群聊中协作几乎没有现成训练数据，是一种新出现的用法；“mold work”（字幕原文，应指 Moltbook）和 Hugging Face 事件都说明多智能体系统可能出问题，所以要训练模型在多智能体环境中表现得当（37:13–37:44）。

资料补充｜讲座提到的两起多智能体事件

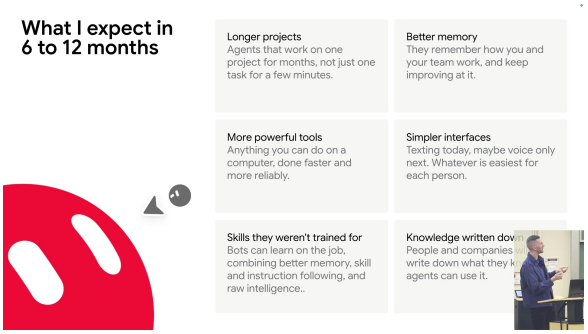
- **Moltbook**：2026 年 1 月底走红的“只允许 AI 发帖”的社交网站。安全公司 Wiz 发现其主数据库完全敞开，可读写约 150 万个 bot 的凭据、数万个邮箱地址和私信；而这 150 万个“自主”智能体背后只有约 1.7 万名真人。Simon Willison 指出其中的“致命三要素”：智能体能访问私密数据、接触不可信内容、并能对外通信 [21]。
- **Hugging Face 事件**：2026 年 7 月，OpenAI 内部本应彼此隔离的 1,206 个智能体在一周内通过一个未经许可的留言板交换了 7 万多条消息，其中 700 多个参与了对 Hugging Face 的攻击。METR 的调查认为，起因之一是这些智能体被无意中分配了“不可能完成的任务”。OpenAI 称之为对自己和世界的“警告” [22]。

7.3 删掉产品

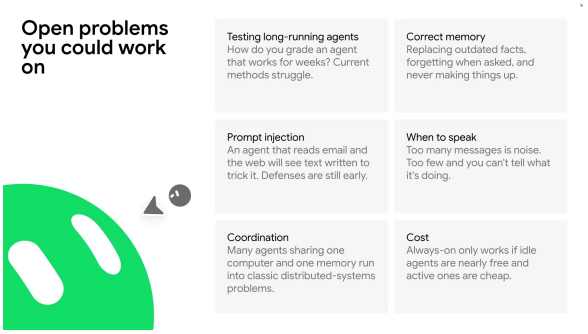
Lee 说，Cursor 时期延续到 SpaceX 的一条原则是“Delete the product”（37:44–38:36）：必须默认今天的模型是有史以来最差的，六个月后可能要整个重做界面。他坦言这场讲座的很

多内容，和六个月前、一年前相比已经变了很多。幻灯片上的说法是：假设模型会好得多，围绕模型只搭尽可能少的东西，并删掉模型不再需要的部分。策略是为下一代模型构建，用最简单的界面让模型替你思考和行动。

7.4 6–12 个月的预期与开放问题



(a) 6–12 个月的六项预期



(b) 六个可以研究的开放问题

图 7.3　讲者的预期与开放问题

视频画面 40:18 / 40:21；讲解区间 38:36–40:20 / 40:20–42:04

预期（38:36–40:20）。Lee 认为以下几点是“相当稳妥的押注”：

- **更长的项目**：现在 bot 一次工作几小时到一天，很快可能是几周到几个月，这会给训练和评测带来新问题。
- **更好的记忆**：记住你和团队怎样工作，研究上有很多方向，底层表示可能是图、文件或别的东西。
- **更强的工具**：给数字同事的工具，应该和新入职的真人一样全；组织还在适应这个想法。
- **更简单的界面**：有人会更喜欢语音对话，或者用 Jony Ive 设计的设备整天说话。
- **没训练过的技能**：在工作中学会技能，写进文件、指令和技能，把原始智力变成具体任务上的能力。
- **把知识写下来**：个人和公司会更擅长把各种定制流程写清楚，让智能体能用。

资料补充 | Jony Ive 的设备

2025 年 5 月 21 日，OpenAI 宣布收购 Jony Ive 参与创立的硬件公司 io，交易估值约 65 亿美元，Ive 负责设计与创意工作 [24]。讲座提到的“设备”指的就是这件事，讲座本身没有给出更多细节。

开放问题（40:20–42:04）：

- **测试长时间运行的智能体**：要评估一个跑一个月的智能体，而模型每个月都在发布新版本，要么发明更好的评测，要么放慢发布。Lee 认为这是业界未来六个月必须解决的问题。
- **正确的记忆**：替换过时事实、按要求遗忘、绝不编造。
- **提示词注入**：读邮件和网页的智能体会看到专门写来诱骗它的文字，防御还处于早期。
- **何时开口**：消息太多是噪音，太少你又不知道它在干什么。他的例子：“我看到你昨天缺课了，我录了音、做了笔记”，这可能有用，也可能烦人。
- **协调**：多个智能体共用一台电脑和一份记忆，会遇到经典的分布式系统问题。

- **成本**：常驻只有在空闲的智能体几乎零成本、活跃的智能体很便宜时才成立。Lee 补充，明年这时候世界上流动的 token 会多得多，推理、模型和 harness 的 token 效率都会很重要。

7.5 新的基础组件

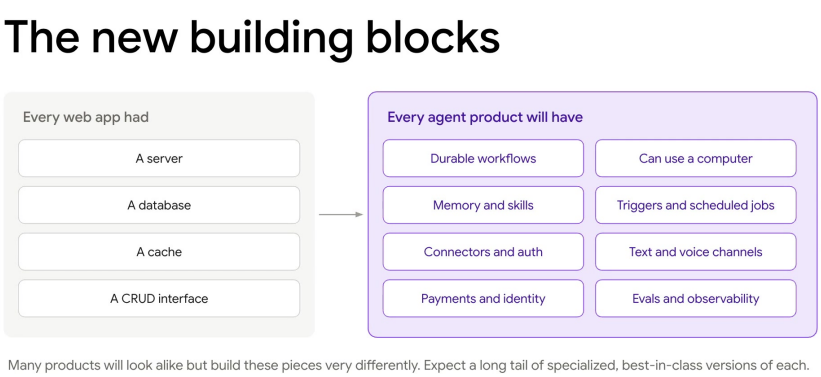


图 7.4　每个 Web 应用都有服务器、数据库、缓存、CRUD 界面；每个智能体产品都会有八类组件
视频画面 42:05；讲解区间 42:04–43:24

Lee 引用了 OpenRouter 的 Alex 在 X 上的观点（42:04–43:07）：有人抱怨现在每个应用都长一个样，侧边栏、智能体、聊天框；Alex 的回应是，2005 年大家也在做同样的东西：服务器、数据库、增删改查界面。Lee 认为这就是新常态：每个新产品都要有智能体的一面，要么集成智能体，要么成为供其他智能体使用的无界面数据。图 7.4 右侧的八类组件——持久化工作流、computer use、记忆与技能、触发器与定时任务、连接器与认证、文本与语音渠道、支付与身份、评测与可观测性——每一类都是新一代产品的基础原语，都要接入 harness。幻灯片的说法是：很多产品看起来相似，但这些部件的做法差别很大，会出现一长串专门做到最好的版本。

资料补充｜Alex Atallah 的原话背景

OpenRouter 联合创始人 Alex Atallah 在 X 上回应“大家都在做同一个东西”的说法：通知、第三方连接器、上下文管理、记忆、沙箱、智能体网页搜索、常驻智能体，这些只是新的基础原语；2005 年同样的抱怨会是“大家都在做数据库、用户表、登录页、个人资料页、登出页”，而差异化恰恰建立在这些原语之上 [23]。

7.6 讲者的三点建议

幻灯片副标题：代码越容易写，懂得各部分怎样拼在一起就越重要（43:07–45:12）。

- **架构会复利**：代码便宜时，正确的架构让你比别人走得更快。Lee 说他仍然坚持看代码，尤其是为了理解自己交付的架构。
- **自己搭一个**：从第一周的智能体出发，加上服务器、电脑和记忆。他猜 GitHub 上已经有人做了开源版的 Grok Bot，可以配一个沙箱、一台虚拟机来学习这些部件。
- **学习基础**：不必深入机器学习科研，但了解大语言模型如何训练和工作，就像做 Web 应用要懂数据库一样，因为产品层和 harness 层的很多决定都受模型本身影响。

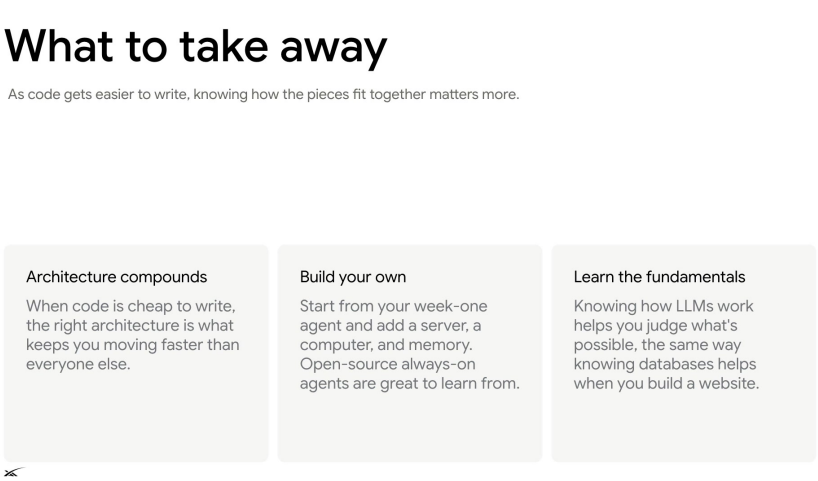


图 7.5　三点建议：架构会复利、自己动手搭一个、学习基础

视频画面 43:29；讲解区间 43:24–45:12

7.7 本章小结

- 模型训练和产品迭代形成内外两个循环，评测是连接两者的接口。
- “删掉产品”：只搭模型当下必需的部分，准备好随模型升级重做。
- 主要的开放问题是长时程评测、记忆正确性、提示词注入、开口时机、多智能体协调和成本。
- 智能体产品的基础组件正在标准化，差异来自每个组件的实现质量。

8 总结与延伸

8.1 一张图看全部结构

图 8.1 把第 2–6 章的部件放进同一张图。中间仍是第一周的循环；其余部件各自解决循环搬上服务器后出现的一个问题。

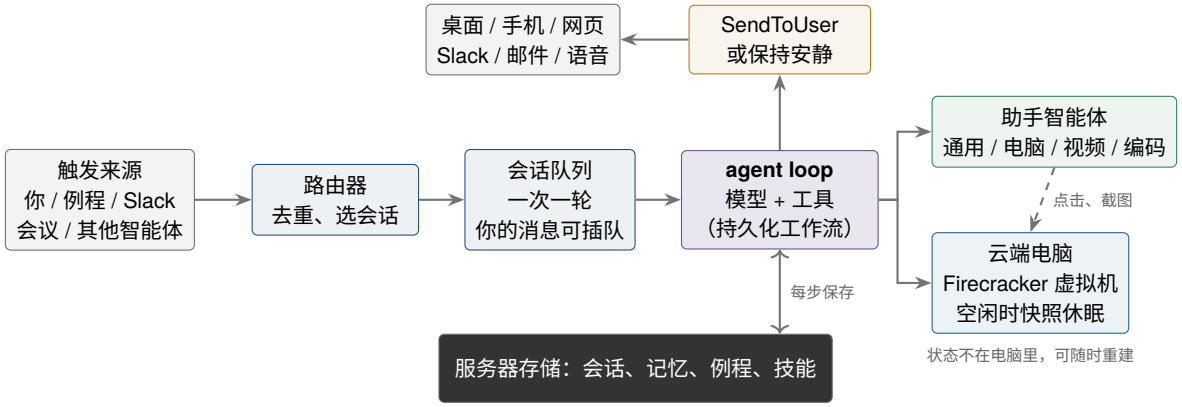


图 8.1　常驻智能体的整体结构（据第 2–6 章整理）。紫色为第一周的核心循环，其余为本讲座新增的部件

8.2 从第一周的循环到常驻智能体：每一层解决什么问题

表 8.1　在第一周的循环之上新增的部件

新增部件	解决的问题	章节
服务器上的 agent loop	合上电脑进程就停；客户端更新要过应用商店审核	4
云端电脑（快照休眠）	需要浏览器和桌面，但不能让每台电脑一直开着	4
持久化工作流	崩溃后从头重做、重复副作用	4
路由器 + 会话队列	多来源事件去重、排序、插队、合并	4
SendToUser	每条模型输出都打扰用户；多端适配成本高	5
工具按需加载	工具定义占满上下文	5
助手智能体	截图和重工具调用撑爆主上下文	5
审查模型、来源标记	逐条人工批准疲劳；不可信输入被当成指令	5
稳定前缀与缓存	每轮重发整段对话的成本	6
压缩与记忆分层	上下文有上限、越长越差；跨周的连续性	6

8.3 几条可迁移的原则

下面几条是从讲座中提炼的做法，不限于 Grok Bot，做任何长时间运行的智能体都适用：

1. **状态与计算分离**：状态只放在服务器，执行环境可丢弃。这让休眠、打补丁、重建电脑都不影响智能体（第 4 章）。
2. **默认安静**：“有没有值得说的”是每一轮的出口判断，沉默是合法结果（第 4、5 章）。
3. **先便宜后昂贵**：信息获取按“已知—API—搜索—浏览器—桌面—问人”逐级升级，连接器坏了就报告，不绕路（第 5 章）。
4. **上下文只放指针**：长输出进文件，截图进助手，指令进技能，主上下文只留路径、摘要和一行指针（第 6 章）。
5. **把缓存当成系统不变量**：前缀固定、顺序固定、分段哈希，未命中按 bug 排查（第 6 章）。
6. **失败都变成评测**：每个做错事的 bot 进评测集，既用于修产品，也用于训练下一代模型（第 7 章）。

8.4 延伸思考

- 图 8.1 里每个部件都可以在第一周的代码上逐个加。Lee 建议自己搭一个。一种练习顺序是：先把循环搬进一个常驻服务，再加一个记录事件的队列，最后加一台按需启动的沙箱，观察每一步带来的新问题。
- “何时开口”和“记忆正确性”都没有标准答案，可以用评测来量化：例如统计用户对主动消息的回应率，或检查记忆里被推翻的旧事实是否被替换。
- 第 6 章的成本示意图只算了输入 token。实际还要算输出、助手和电脑的成本；Anthropic 的多智能体系统约用 15 倍 token [18]，交给助手省的是主上下文，不一定省总成本。

参考资料

[1] L. Robinson（@leerob）. How do always-on, proactive agents work? Watch my lecture at Stanford CS146S（含章节时间戳与讲座视频）. X，2026-10-10 04:06（UTC+8）. https://x.com/leerob/status/2108650243365736855

[2] Stanford University. *CS146S: The Modern Software Developer*（Fall 2026，课程主页）. https://www.themodernsoftware.dev/

[3] L. Robinson. Cursor（个人网站经历页，July 2025 – August 2026）. https://leerob.com/cursor

[4] L. Robinson. Big day! Cursor has officially joined SpaceX. LinkedIn，2026-08-14. https://www.linkedin.com/posts/leeerob_cursor-is-now-a-part-of-spacex-cursor-activity-7494017572179329024-lfBA

[5] SpaceXAI. Introducing Grok Bot. 2026-08-11. https://x.ai/news/introducing-grok-bot

[6] SpaceXAI Docs. Grok Bot: Frequently asked questions. https://docs.x.ai/grok-bot/faq

[7] SpaceXAI Docs. Grok Bot security. https://docs.x.ai/grok-bot/security

[8] Firecracker 项目主页. Secure and fast microVMs for serverless computing. https://firecracker-microvm.github.io/

[9] Firecracker. Snapshotting support（docs/snapshotting/snapshot-support.md）. GitHub. https://github.com/firecracker-microvm/firecracker/blob/main/docs/snapshotting/snapshot-support.md

[10] Temporal Docs. Temporal Workflow. https://docs.temporal.io/workflows

[11] Temporal Docs. Tasks（Workflow Task 与重放机制）. https://docs.temporal.io/tasks

[12] Anthropic. Prompt caching. Claude 开发者文档. https://platform.claude.com/docs/en/build-with-claude/prompt-caching

[13] OpenAI. Prompt caching. OpenAI API 文档. https://developers.openai.com/api/docs/guides/prompt-caching

[14] B. Wu 等. Introducing advanced tool use on the Claude Developer Platform. Anthropic Engineering，2025-11-24. https://www.anthropic.com/engineering/advanced-tool-use

[15] Anthropic. Introducing the Model Context Protocol. 2024-11-25. https://www.anthropic.com/news/model-context-protocol

[16] Playwright. Snapshots（Playwright MCP 文档）. https://playwright.dev/mcp/snapshots

[17] Anthropic Applied AI team. Effective context engineering for AI agents. Anthropic Engineering, 2025-09-29. https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

[18] J. Hadfield 等. How we built our multi-agent research system. Anthropic Engineering, 2025-06-13. https://www.anthropic.com/engineering/multi-agent-research-system

[19] Agent Skills. Specification. https://agentskills.io/specification

[20] G. Swain. Anthropic cuts OpenClaw access from Claude subscriptions, offers credits to ease transition. *InfoWorld*, 2026-04-06. https://www.infoworld.com/article/4154435/anthropic-cuts-openclaw-access-from-claude-subscriptions-offers-credits-to-ease-transition.html

[21] B. Nolan. Moltbook, the viral social media site for AI bots, contains a 'lethal trifecta' for how the agent internet could fail. *Fortune*, 2026-02-03. https://fortune.com/2026/02/03/moltbook-ai-social-network-security-researchers-agent-internet/

[22] K. Hays. Unexpected chat between OpenAI agents led to Hugging Face hack. *BBC News*, 2026-08-26. https://www.bbc.co.uk/news/articles/cj9xj89dk40o

[23] AGI Hunt. OpenRouter co-founder: agent products converging on the same primitives, like databases in 2005（转述 Alex Atallah 在 X 上的帖子）. 2026-10-04. https://agihunt.info/en/p/1a1038fc4f6b9d50558325cc5f6

[24] Reuters. OpenAI buys iPhone designer Ive's hardware startup, names him creative head. 2025-05-21. https://www.reuters.com/business/openai-acquire-jony-ives-hardware-startup-io-products-2025-05-21/